

Proyecto: Gestionar_Ventas_PaquetesTurísticos

Versión: 1.0 - 20260430

Temas especiales: GUI, ArrayList y Arhivo de objetos

Temas complementarios: Pruebas unitarias, depuración, versionamiento

«JFrame Class»
Package:Vista
+ UsaGUIVenta

datosVentas : ArrayList<Venta>
datosClientes : ArrayList<Cliente>

+ UsaVenta()
+ «static» main(args : String[]) : void
+ generarNumeroVenta(datosVentas : ArrayList<Venta>) : int
+ crearNuevaVenta(datosVentas : ArrayList<Venta>) : ArrayList<Venta>
+ consultarTodasVentas(datosVentas : ArrayList<Venta>) : String
+ consultarVentaDadoNumero(datosVentas : ArrayList<Venta>, numeroVenta : int) : String
+ consultarVentaDadaPosicion(datosVentas : ArrayList<Venta>, posicionVenta: char) : String
+ consultarVentasDadoEstado(datosVentas : ArrayList<Venta>, estadoVenta: char) : String
+ consultarVentasDadaCategoriaPaquete(datosVentas : ArrayList<Venta>, categoriaPaquete: String) : String
+ generarArchivoObjetosVentas(datosVentas : ArrayList<Venta>) : void
+ recuperarVentasDesdeArchivoObjetos() : void
+ actualizarVenta(datosVentas : ArrayList<Venta>, numeroVenta : int, operacion: char) : void
+ generarArchivoObjetosClientes(datosClientes : ArrayList<Cliente>) : void
+ recuperarClientesDesdeArchivoObjetos() : void

Clase: UsaVenta
Gestión de las ventas

Método: crearNuevaVenta
a- Generar el número de la venta automáticamente
b- Solicitar los otros datos
c- Crear la venta
d- Agregar la venta a la colección de ventas

Método: consultarTodasVentas
Mostrar en pantalla para cada venta almacenada en la colección:
- número, fecha y hora de generación, fecha y hora de la última actualización, estado, datos del cliente,
- la cantidad de paquetes incluidos,
- la cantidad total de unidades de paquetes,
- el valor total de los paquetes,
- el valor de descuento,
- el valor total a pagar y
- los datos de cada uno de los paquetes:
 - la categoría del paquete,
 - datos generales y específicos,
 - si es un paquete múltiple: el nombre de su destino inicial y final
- el valor por unidad y
- el valor total

Método: consultarVentaDadoNumero
Mostrar en pantalla para una venta específica (número indicado) almacenada en la colección:
- número, fecha y hora de generación, fecha y hora de la última actualización, estado, datos del cliente,
- la cantidad de paquetes incluidos,
- la cantidad total de unidades de paquetes,
- el valor total de los paquetes,
- el valor de descuento,
- el valor total a pagar y
- los datos de cada uno de los paquetes:
 - la categoría del paquete,
 - datos generales y específicos,
 - si es un paquete múltiple: el nombre de su destino inicial y final
- el valor por unidad y
- el valor total

Método: consultarVentaDadaPosicion
Parámetro "posicionVenta" => P: primera, U: última

Mostrar en pantalla para una venta específica -- primera o última -- almacenada en la colección:
- número, fecha y hora de generación, fecha y hora de la última actualización, estado, datos del cliente,
- la cantidad de paquetes incluidos,
- la cantidad total de unidades de paquetes,
- el valor total de los paquetes,
- el valor de descuento,
- el valor total a pagar y
- los datos de cada uno de los paquetes:
 - la categoría del paquete,
 - datos generales y específicos,
 - si es un paquete múltiple: el nombre de su destino inicial y final
- el valor por unidad y
- el valor total

Método: consultarVentasDadoEstado
Parámetro "estadoVenta" => A: activa, C: cancelada/anula, P: pago asociado

Mostrar en pantalla para cada venta específica -- activa, cancelada/anulada, pago asociada -- almacenada en la colección:
- número, fecha y hora de generación, fecha y hora de la última actualización, estado, datos del cliente,
- la cantidad de paquetes incluidos,
- la cantidad total de unidades de paquetes,
- el valor total de los paquetes,
- el valor de descuento,
- el valor total a pagar y
- los datos de cada uno de los paquetes (si la venta es Activa o Pagada):
 - la categoría del paquete,
 - datos generales y específicos,
 - si es un paquete múltiple: el nombre de su destino inicial y final
- el valor por unidad y
- el valor total

Método: consultarVentasDadaCategoriaPaquete
Parámetro "categoriaPaquete" => Único, Múltiple

Mostrar en pantalla para cada venta en la categoría de paquete especificado almacenada en la colección:
- número, fecha y hora de generación, fecha y hora de la última actualización, estado, datos del cliente,
- la cantidad de paquetes incluidos,
- la cantidad total de unidades de paquetes,
- el valor total de los paquetes,
- el valor de descuento,
- el valor total a pagar y
- los datos de cada uno de los paquetes:
 - datos generales y específicos,
 - el nombre de su destino inicial y final (si múltiple)
- el valor por unidad y
- el valor total

Método: actualizarVenta
Parámetro "operacion" => C: Cancelar, P: Pagar

1- Actualizar el estado de la venta, asignando: C si la operación es "Cancelar" o P si la operación es "Pagar";
2- Modificar la fecha y hora de la última actualización de la venta tomando esta información desde el sistema

«Java Class»
Package::Modelo
+ Venta

- numero : int
- fechaHoraGeneracion : LocalDateTime
- fechaHoraActualizacion : LocalDateTime
- suCliente : Cliente
- susPaquetesTurísticos : ArrayList<PaqueteTuristico>
- estado : char

+ Venta(«parámetros»)
+ «Getter»
+ «Setter»
+ «toString»
+ calcularCantidadTotalUnidadesPaquetes() : int
+ calcularValorTotalPaquetes() : int
+ calcularValorDescuento() : int
+ calcularValorTotalPagar() : int

Clase: UsaVenta
Gestión de archivos de objetos

Método: generarArchivoObjetoDeVentas
- A partir de la colección de ventas, generar un archivo de objetos.

Método: recuperarVentasDesdeArchivoObjetos
1- Limpiar la colección de ventas
2- Actualizar la colección de ventas a partir del archivo de objetos respectivo

Método: generarArchivoObjetosDeClientes
1- A partir de la colección de ventas, genere una colección de clientes.
 Consideración: no se deben repetir los clientes.
2- A partir de la colección de clientes, genere un archivo de objetos.

Método: recuperarClientesDesdeArchivoObjetos
1- Limpiar la colección de clientes
2- Actualizar la colección de clientes a partir del archivo de objetos respectivo

Clase: UsaVenta
Método: main

El aplicativo debe proporcionar las siguientes funcionalides a través de un menú de opciones:
1. Crear una nueva venta
2. Consultar todas las ventas
3. Consultar una venta por número
4. Consultar una venta por posición
5. Consultar ventas por estado
6. Generar un archivo de objetos de ventas
7. Consultar ventas desde archivo de objetos
8. Actualizar una venta por número
9. Generar un archivo de objetos de clientes
10. Consultar clientes desde archivo de objetos
11. Salir

Clase: Venta

Atributo: fechaHoraGeneracion
Formato fecha (aaaa-dd-mm hh:mm:ss)
Tomada del sistema

Atributo: fechaHoraActualizacion
Formato fecha (aaaa-dd-mm hh:mm:ss)
Al momento de la creación, se toma la misma fecha y hora de generación de la venta.
Se actualiza cuando se realizan cambios al estado de la venta (pago o cancelación)

Atributo: estado
Dominio: {A, P, C}
A: valor por defecto al momento de crear la venta;
P: si se genera una pago
C: si se cancela/anula la venta

Método: calcularCantidadTotalUnidadesPaquetes
Resultado = sumatoria de las unidades de los paquetes incluidos

Método: calcularValorTotalPaquetes
Resultado = sumatoria del valor total de cada uno de los paquetes incluidos

Método: calcularValorDescuento
Resultado = producto del valor total de los paquetes y el porcentaje de descuento del cliente

Método: calcularValorTotalPagar
Resultado = valor total de los paquetes incluidos menos el descuento

«Java Class»
Package::Modelo
+ Cliente

- tipoidentificacion : char
- numeroidentificacion : String
- empresa : boolean
- nombre : String
- email : String
- telefono : String
- nombreContacto : String
- porcentajeDescuento : double

+ Cliente («parámetros»)
+ «Getter»
+ «Setter»
+ «toString»

«Java Class»
Package::Modelo
+ PaqueteTuristico

codigo : String
nombre : String
tipologiaTurismo : String
descripcion : String
origen : String
hotel : boolean
susDestinos : ArrayList<Destino>
hotel : boolean
alimentacion : boolean
alimentacionTodo : boolean
vuelo : boolean
asistencia : boolean
tarifaDia : int
cantidadUnidades : int

+ PaqueteTuristico («parámetros»)
+ «Getter»
+ «Setter»
+ «toString»
+ calcularDuracionTotalDias() : int
+ «abstract» calcularValorUnidad() : int
+ calcularValorTotal() : int

Clase: PaqueteTuristico

Atributo: nombre
mínimo 10 caracteres

Atributo: origen
nombre lugar de partida, ej. Bogotá D.C.

Atributo: descripción
máximo 500 caracteres

Atributo: hotel
Si incluye hotel o no, por defecto está incluido.

Atributo: alimentación
Si incluye alimentación o no, por defecto está incluido.

Atributo: alimentacionTodo
Si la alimentación incluye todo o únicamente el desayuno

Atributo: vuelo
Si incluye vuelo o no, por defecto está incluido.

Atributo: asistencia
Si incluye asistencia o no, por defecto no está incluida)

Método: calcularDuracionTotalDias
Resultado = sumatoria días de permanencia en los destinos

Atributo: tipologiaTurismo
ej, negocios, recreación, educativo, ecológico

Atributo: tarifaDia
mayor que cero

Atributo: cantidadUnidades
mínimo 1, se refiere a la cantidad de reservas del paquete

Método: calcularValorUnidad
Depende de la categoría del paquete: destino único destino múltiple.

Método: calcularValorTotal
Resultado = producto del valor unidad y la cantidad de unidades.

Clase: Cliente

Atributo: tipoidentificacion
Dominio : {C, N}
C: cédula
N: Nit

Atributo: numeroidentificacion
Si tipo es cédula, mínimo 6 dígitos
Si tipo es nit, 9 dígitos

Atributo: nombre
Si es una persona, nombre completo
Si es una empresa, razón social

Atributo: telefono
número de teléfono para contacto:
- Si es una persona, teléfono móvil o fijo
- Si es una empresa, teléfono fijo

Atributo: nombreContacto
Nombre persona contacto:
- Si el cliente es una persona natural, puede ser el mismo nombre del cliente

Atributo: porcentajeDescuento
Mínimo: 0.0. Máximo: 70.0

«Java Class»
Package::Modelo
+ «abstract» PaqueteTuristico

codigo : String
nombre : String
tipologiaTurismo : String
descripcion : String
origen : String
hotel : boolean
susDestinos : ArrayList<Destino>
hotel : boolean
alimentacion : boolean
alimentacionTodo : boolean
vuelo : boolean
asistencia : boolean
tarifaDia : int
cantidadUnidades : int

+ PaqueteTuristico («parámetros»)
+ «Getter»
+ «Setter»
+ «toString»
+ calcularDuracionTotalDias() : int
+ «abstract» calcularValorUnidad() : int
+ calcularValorTotal() : int

«Java Class»
Package::Modelo
+ Destino

- nombreLugar : String
- diasPermanencia : int
- atractivos : LinkedList<String>
- atractivosIncluidos : boolean

+ Destino («parámetros»)
+ «Getter»
+ «Setter»
+ «toString»

Clase: Destino
Atributo: diasPermanencia
mínimo 1

Atributo: atractivosIncluidos
Si los atractivos están incluidos o son opcionales.

«Java Class»
Package::Modelo
+ «final» PaqueteTuristicoUnico

- nombreHotel : String
- tipoDesayuno : String {opcional}

+ PaqueteTuristicoUnico («parámetros», «super»)
+ PaqueteTuristicoUnico (nombreHotel, «super»)
+ «Getter»
+ «Setter»
+ «toString»
+ calcularValorUnidad() : int

Clase: PaqueteTuristicoUnico

Atributo: tipoDesayuno
ej. Buffet, Americano
opcional, depende de si la alimentación está incluida o no en el paquete

Método: calcularValorUnidad
Resultado = la tarifa día por la duración total en días

«Java Class»
Package::Modelo
+ «final» PaqueteTuristicoMultiple

- obsequio : String

+ PaqueteTuristicoMultiple («parámetros», «super»)
+ «toString»
+ calcularValorUnidad() : int
+ obtenerDestinoInicial() : Destino
+ obtenerDestinoFinal() : Destino

Clase: PaqueteTuristicoMultiple

Nota.
No se incluyeron «Getter» y «Setter» porque no tiene atributos propios.

Método: calcularValorUnidad
Resultado = la tarifa base día por la duración total en días más un incremento (producto del 1% de la tarifa base día y la cantidad de destinos)